

Reviewer Pack — Minimal Order of Quadratic Residues in rings and DPLL Proof ...

Entry eadd43cabe3d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 21:01:27 UTC

Minimal Order of Quadratic Residues in rings and DPLL Proof Length

Entry ID: eadd43cabe3d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 21:01:27 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory (Quadratic Residue Symbol) **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

Statement:

The length of the shortest DPLL proof for a CNF φ is upper bounded by the order of the quadratic residue symbol modulo some prime q that divides the field size of φ 's variables, i.e., $E[\text{dpll}(\varphi)] \leq \Theta(\log(q))$ where q is the smallest prime dividing $|\text{Var}(\varphi)|$.

Rationale (proposer's reasoning):

Quadratic residues provide a measure of the algebraic structure in the coefficient field of a CNF. A deeper understanding of how this structure influences proof length could shed light on the inherent complexity of SAT solving. The use of quadratic residues as an invariant for DPLL proof length has not been explored before.

Taxonomy category: algebraic_number_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9e13e1675d239711

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated CNFs, the shortest DPLL proof length is less than or equal to the calculated $\log(q)$ value, where q is the smallest prime dividing $|\text{Var}(\varphi)|$, with $p\%$ of the seeds showing a correlation coefficient greater than 0.8 and an aggregate mean metric value of $\log(q)$ not exceeding the median by more than 3 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - quadratic residue symbol AND DPLL proof length - DPLL complexity AND algebraic number theory quadratic residues - algebraic number theory modulo prime AND DPLL proof upper bound

Top relevant hits considered: - [http://arxiv.org/abs/2011.11054v2] Consecutive Quadratic Residues And Quadratic Nonresidue Modulo p - [http://arxiv.org/abs/2304.07644v1] Chain Lemma, Quadratic Forms and Symbol Length - [http://arxiv.org/abs/2405.13159v2] Small Prime k th Power Residues and Nonresidues in Arithmetic Progressions - [http://arxiv.org/abs/math/0404268v2] Simultaneous approximation by conjugate algebraic numbers in fields of transcendence degree one - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/1510.03465v2] A Simple Proof Of The Prime Number Theorem - [http://arxiv.org/abs/0709.0640v1] Algebraic theta functions and Eisenstein-Kronecker numbers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def smallest_prime_dividing(n):
        for i in range(2, n + 1):
            if n % i == 0 and is_prime(i):
                return i
        return None

    def dpll(clauses, assignment={}):
        if not clauses:
            return True
```

```

unit_clauses = [c[0] for c in clauses if len(c) == 1]
pure_symbols = {}
for symbol in set(symbol for clause in clauses for symbol in clause):
    positive_count = sum(1 for clause in clauses if symbol in clause)
    negative_count = sum(1 for clause in clauses if -symbol in clause)
    if positive_count == 0:
        pure_symbols[symbol] = True
    elif negative_count == 0:
        pure_symbols[symbol] = False

for symbol, value in pure_symbols.items():
    if not dpll([c for c in clauses if symbol not in c and -symbol not in c], assignment | {symbol: value}):
        continue
    return True

literal = unit_clauses[0]
if literal > 0:
    if not dpll(clauses, assignment | {literal: True}):
        return dpll(clauses, assignment | {literal: False})
else:
    if not dpll(clauses, assignment | {-literal: True}):
        return dpll(clauses, assignment | {-literal: False})

return False

def generate_cnf(n):
    clauses = []
    for _ in range(2 * n):
        clause = [random.randint(-n, -1), random.randint(1, n)]
        if random.choice([True, False]):
            clause.reverse()
        clauses.append(clause)
    return clauses

def log_q(q):
    return math.log(q)

n_max = 0
instances_tested = 0
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for _ in range(30):
    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    q = smallest_prime_dividing(n)
    if q is None:
        continue

    instances_tested += 1
    n_max = max(n_max, n)

    proof_length = dpll(cnf)
    if not proof_length:
        counterexample = f"CNF with {n} variables has no DPLL proof"

```

```

        conjecture_holds = False
        break

    metric_value = log_q(q)
    total_metric_value += metric_value

mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0.0
std_dev = math.sqrt(sum((x - mean_metric_value) ** 2 for x in [log_q(smallest_prime_dividing(r)) for r in results]))

return {
    "metric_name": "DPLL Proof Length",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / (len(results) - 1))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8 and std_dev <= 3 * std_dev:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5ad250e.py", line 118, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5ad250e.py", line 91, in run_trial
    proof_length = dpll(cnf)
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5ad250e.py", line 49, in dpll

```


2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eadd43cabe3d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eadd43cabe3d.tar.gz` (if generated)